

Week 4. Homework 2. SQL-II

Submission Guidelines

- **Dataset:** You must use the provided *IMDB.sqlite* dataset to answer all questions.
- **Code and Output:** Copy your code and attach screenshots of your outputs under each question.
- **Submission Format:** Each student is required to push one Word or PDF file to a designated GitHub repository you create specifically for this week's assignment.
- **Repository Name:** Use the format *Week 4-HW2-LastName*, where *LastName* is your last name.
- **Code Quality:** Ensure your code is organized for easy understanding.
- **AI Usage:** You should make a serious attempt at each problem prior to using an LLM. Remember, you may be tested on SQL knowledge without assistance in a technical interview. If prompting an LLM, I suggest telling it your starting point, your thought process, and asking it to guide you. I've done this learning SQL and it is helpful and I learned more that way. Disclose your usage at the end of the file; you will not be penalized regardless, but this give me some information.

Questions

Date-Based Queries

1. Find the First and Last Movie Released Each Year

Retrieve the first and last movie released in each year. Display the year, first movie title, and last movie title.

```
SELECT strftime('%Y', release_date) AS year,  
FIRST_VALUE(title) OVER (  
PARTITION BY strftime('%Y', release_date)  
ORDER BY release_date  
) AS first_movie,  
FIRST_VALUE(title) OVER (  
PARTITION BY strftime('%Y', release_date)  
ORDER BY release_date DESC  
) AS last_movie  
FROM movies  
GROUP BY year;
```

	year	first_movie	last_movie
1	1916	Intolerance	Intolerance
2	1925	The Big Parade	The Big Parade
3	1927	Metropolis	Metropolis
4	1929	Pandora's Box	The Broadway Melody
5	1930	Hell's Angels	Hell's Angels
6	1932	A Farewell to Arms	A Farewell to Arms
7	1933	42nd Street	She Done Him Wrong

Execution finished without errors.
Result: 90 rows returned in 552ms

2. Total Revenue Generated Each Year

Calculate the total revenue generated by movies released in each year. Display the year and total revenue, ordered by revenue in descending order.

```
SELECT strftime('%Y', release_date) AS year, SUM(revenue) AS total_revenue
FROM movies
GROUP BY year
ORDER BY total_revenue DESC;
```

	year	total_revenue
1	2012	24141710246
2	2014	24120490589
3	2013	23411493295
4	2015	22775024221
5	2009	21072651506
6	2011	20516921160
7	2010	20348574768

Execution finished without errors.
Result: 90 rows returned in 12ms

3. Movies Released in the First Half of the Year

Find all movies that were released between January and June. Display the movie title, release date, and director name.

```
SELECT m.title, m.release_date, d.name AS director_name
```

```
FROM movies m
JOIN directors d ON m.director_id = d.id
WHERE CAST(strftime('%m', release_date) AS INTEGER) BETWEEN 1 AND 6;
```

	title	release_date	director_name
1	Pirates of the Caribbean: At World's...	2007-05-19	Gore Verbinski
2	John Carter	2012-03-07	Andrew Stanton
3	Spider-Man 3	2007-05-01	Sam Raimi
4	Avengers: Age of Ultron	2015-04-22	Joss Whedon
5	Batman v Superman: Dawn of Justice	2016-03-23	Zack Snyder
6	Superman Returns	2006-06-28	Bryan Singer
7	Pirates of the Caribbean: Dead Man's...	2006-06-20	Gore Verbinski

Execution finished without errors.
Result: 2166 rows returned in 10ms

4. Number of Movies Released on Weekends

Count the total number of movies released on a weekend (Saturday or Sunday). Use SQLite's strftime('%w') to determine the day of the week (0 = Sunday, 6 = Saturday).

```
SELECT COUNT(*) AS weekend_movies
FROM movies
WHERE strftime('%w', release_date) IN ('0','6');
```

	weekend_movies
1	451

Execution finished without errors.
Result: 1 rows returned in 17ms

CASE Statement Queries

5. Categorize Directors Based on Total Revenue

WEEK 4 · HOMEWORK 2 SQL-II

Classify directors based on the total revenue of their movies:

- High Revenue if total revenue > \$1 billion
- Moderate Revenue if total revenue is between \$500 million and \$1 billion
- Low Revenue if total revenue < \$500 million

```
SELECT d.name AS director_name,
SUM(m.revenue) AS total_revenue,
CASE
WHEN SUM(m.revenue) > 1000000000 THEN 'High Revenue'
WHEN SUM(m.revenue) >= 500000000 THEN 'Moderate Revenue'
ELSE 'Low Revenue'
END AS revenue_category
FROM movies m
JOIN directors d ON m.director_id = d.id
GROUP BY d.id;
```

Find in editor ☐ Case Sensitive ☐ Whole Words ☐ Regular Expression			
	director_name	total_revenue	revenue_category
1	James Cameron	5883569439	High Revenue
2	Gore Verbinski	3177013357	High Revenue
3	Sam Mendes	2713548809	High Revenue
4	Christopher Nolan	4227483234	High Revenue
5	Andrew Stanton	2109045355	High Revenue
6	Sam Raimi	3132901462	High Revenue
7	Byron Howard	591794936	Moderate Revenue

Execution finished without errors.
Result: 3349 rows returned in 12ms

6. Movies Based on Popularity and Revenue

Classify movies into the following categories based on popularity and revenue:

- Blockbuster if popularity > 100 and revenue > \$500 million
- Hit if popularity > 50 and revenue between \$100 million and \$500 million
- Average for all other movies

```
SELECT title, popularity, revenue,
CASE
WHEN popularity > 100 AND revenue > 500000000 THEN 'Blockbuster'
WHEN popularity > 50 AND revenue BETWEEN 100000000 AND 500000000
THEN 'Hit'
ELSE 'Average'
```

END AS category

FROM movies;

	title	popularity	revenue	category
1	Avatar	150	2787965087	Blockbuster
2	Pirates of the Caribbean: At World's...	139	961000000	Blockbuster
3	Spectre	107	880674609	Blockbuster
4	The Dark Knight Rises	112	1084939099	Blockbuster
5	John Carter	43	284139100	Average
6	Spider-Man 3	115	890871626	Blockbuster
7	Tangled	48	591794936	Average

Execution finished without errors.
Result: 4773 rows returned in 9ms

7. Directors with Critically Acclaimed and Poorly Rated

Movies List directors and count how many of their movies fall into these categories:

- Critically Acclaimed if vote_average >= 8
 - Poorly Rated if vote_average < 5

```
SELECT d.name AS director_name,  
SUM(CASE WHEN m.vote_average >= 8 THEN 1 ELSE 0 END) AS critically_acclaimed,  
SUM(CASE WHEN m.vote_average < 5 THEN 1 ELSE 0 END) AS poorly_rated  
FROM movies m  
JOIN directors d ON m.director_id = d.id  
GROUP BY d.id;
```

	director_name	critically_acclaimed	poorly_rated
1	James Cameron	0	0
2	Gore Verbinski	0	0
3	Sam Mendes	0	0
4	Christopher Nolan	5	0
5	Andrew Stanton	0	0
6	Sam Raimi	0	0
7	Byron Howard	0	0

Execution finished without errors.
Result: 2349 rows returned in 15ms

For AI I used chatGPT if there was an error in my code that I could not figure out.